

Name: Mark Magdy Ayoub

Id: 2205040

We will test a log file "access.log" that exist in my github link

Record Analysis

First of all we will analyze a random sample record

```
83.149.9.216 - - [17/May/2015:10:05:59 +0000] "GET /presentations/logstash-monitorama-2013/images/logstashbook.png HTTP/1.1" 200 54662
"http://semicomplete.com/presentations/logstash-monitorama-2013/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"
```

That is an apache log record

The anatomy of the record is as follows:

IP - - [timestamp] "HTTP Method URL HTTP/Version" Status_Code Size Referer User-Agent

- + **83.149.9.216**: The IP address of the client making the request
 - + **-** : Not used or not available.
 - + **[17/May/2015:10:05:59 +0000]**: The date and time of the request
 - + **"GET /presentations/logstash-monitorama-2013/images/logstashbook.png HTTP/1.1"**: request
 - **Method**: Get Http Method
 - **Resource**: /presentations/logstash-monitorama-2013/images/logstashbook.png
 - **Protocol**: Http version 1.1
 - + **200**: status code
 - + **54662**: The size of the response in bytes.
 - + **<http://semicomplete.com/presentations/logstash-monitorama-2013/>** : Referer
 - + which means the page from which the request was made
 - + **"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_9_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/32.0.1700.77 Safari/537.36"** : user agent and that include some information about the browser and user's operating system used by the client
-

Code

First we need to define a variable that hold the path of the log file

```
LOG_FILE="access.log"
```

```
[1] number of request
```

Each record represent a request so all what we need to compute how many lines in this file

Using word count cli tool

```
total_requests=$(wc -l < "$LOG_FILE")  
echo "Total number of requests: $total_requests"
```

[2] we need to compute how many get and posts requests

All we need to search on the keyword "GET" and then compute how many lines in the result

And the same thing for "POST"

```
get_count=$(grep -o "GET" "$LOG_FILE" | wc -l)  
post_count=$(grep -o "POST" "$LOG_FILE" | wc -l)  
echo "Number of GET requests: $get_count"  
echo "Number of POST requests: $post_count"
```

[3] compute how many unique ip address

We need awk tool to process text by fields

awk 'print \$1' "\$LOG_FILE" → print the first field which is the ip address

and then sort the result

```
awk 'print $1' "$LOG_FILE" | sort
```

and then take distinct ip to remove duplicates

```
awk 'print $1' "$LOG_FILE" | sort | uniq
```

and then compute how many lines

```
awk 'print $1' "$LOG_FILE" | sort | uniq | wc -l  
unique_ips=$(awk '{print $1}' "$LOG_FILE" | sort | uniq | wc -l)  
echo "Unique IP addresses: $unique_ips"
```

[4] compute get and post per ip

First we need a format like that

Ip address : Total=sum get and post , GET= , POST=

And will be done by awk tool by defining associative arrays that track stats per ip

```
echo "GET and POST count per IP:"
```

```

awk '
{
    ip[$1]++;      # Count total requests per IP

    if ($6 == "GET") get[$1]++; # Count GET requests per IP , $6 variable store the 6th field
    if ($6 == "POST") post[$1]++; # Count POST requests per IP
}
END {
    for (i in ip) {
        printf "%s: Total=%d, GET=%d, POST=%d\n", i, ip[i], get[i]+0, post[i]+0;
    }
}' "$LOG_FILE"

```

[5] Compute Failed Requests

First we need to take the 9th field which is the HTTP Status code and check if it matches status code starts with 4 and 5 (client/server errors) using regular expressions.

The percentage will be computed by taking total requests and divide it by failed requests

```
failed_requests=$(awk '$9 ~ /^[45]/ {count++} END {print count+0}' "$LOG_FILE")
```

```
failure_percentage=$(awk -v total="$total_requests" -v failed="$failed_requests" 'BEGIN {printf "%.2f%%", (failed / total) * 100}')
```

```
echo "Failed requests (4xx or 5xx): $failed_requests"
```

```
echo "Percentage of failed requests: $failure_percentage"
```

[6] Most Active IP Address

All we want to need is to compute the frequency of each ip and sort them in a reverse order to get the highest (using awk tool) and take the first line from the result (using head command)

```
most_active_ip=$(awk '{ip[$1]++} END {for (i in ip) print ip[i], i}' "$LOG_FILE" | sort -nr | head -n1)
```

```
echo "Most active IP address: $most_active_ip"
```

[7] daily request average

That is done by taking the date only from the timestamp field and count how many dates we have
(in summary: the frequency for each date)

Takes the 4th field which is the timestamp ([17/May/2015:10:05:03 +0000])

And extract the date using cut command (cut -d: -f1) [17/May/2015]

And then count how many dates we have (uniq -c)

```
daily_requests=$(awk '{date[$4]=$date[$4]+1} END {for (d in date) print d}' "$LOG_FILE" | cut -d: -f1  
| sort | uniq -c)
```

```
total_days=$(echo "$daily_requests" | wc -l)
```

```
average_daily_requests=$(awk -v total="$total_requests" -v days="$total_days" 'BEGIN {printf  
"%0.2f", total / days}')
```

```
echo "Average daily requests: $average_daily_requests"
```

[8] Failure analysis

This done by substr() method that gets the date portation and sort them

```
echo "Failures per day:"
```

```
awk '$9 ~ /^[45]/ {day=substr($4,1,11); fail[day]++} END {for (d in fail) print fail[d], d}' "$LOG_FILE" |  
sort -nr
```

[9] Hourly Request Count

This done by substr() method that gets the hours portation and sort them

```
echo "Requests by hour:"
```

```
awk '{hour=substr($4,14,2); hourly[hour]++} END {for (h in hourly) print h, hourly[h]}' "$LOG_FILE" |  
sort -n
```

[10] status code frequency

That is simple by compute the frequency of each status code using awk tool

Take the 9th field that represent the status code and compute its frequency then sort them

```
echo "Status code breakdown:"
```

```
awk '{status[$9]++} END {for (s in status) print s, status[s]}' "$LOG_FILE" | sort -n
```

[11] **most active user use get method and top user uses post method**

Take the frequency of the 6th field which is the http method and count them per ip

```
echo "Top IP using GET:"
```

```
awk '$6 == "GET" {ips[$1]++} END {for (ip in ips) print ips[ip], ip}' "$LOG_FILE" | sort -nr | head -n1
```

```
echo "Top IP using POST:"
```

```
awk '$6 == "POST" {ips[$1]++} END {for (ip in ips) print ips[ip], ip}' "$LOG_FILE" | sort -nr | head -n1
```

[12] **failure by hours**

We will look for failure code again but group them by hour

```
echo "Failure requests by hour:"
```

```
awk '$9 ~ /^[45]/ {hour=substr($4,14,2); fail_hour[hour]++} END {for (h in fail_hour) print h, fail_hour[h]]}' "$LOG_FILE" | sort -n
```

the output will be in the output.txt